

Invoking Business Processes



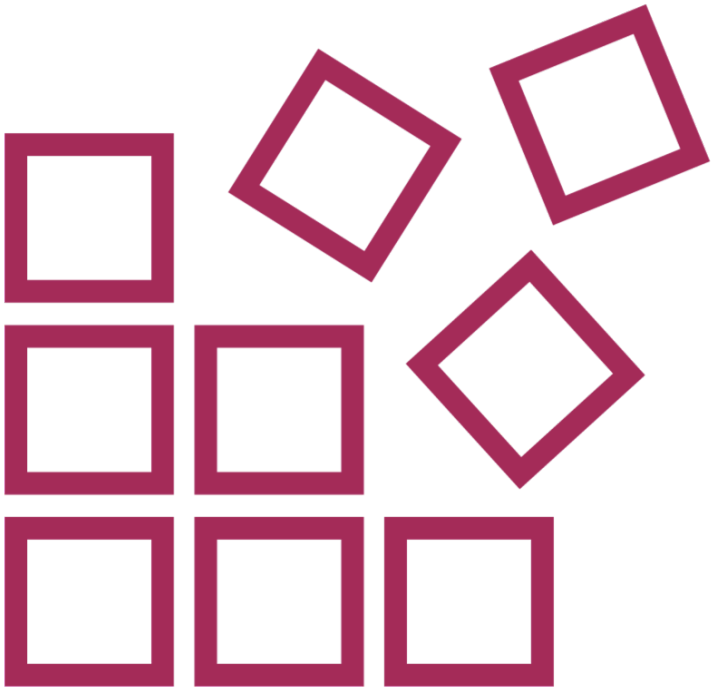
Michael L Perry

SOFTWARE MATHEMATICIAN

@michaelperry immutablearchitecture.com



Making the Pieces Work Together



Process

- Steps
- Rules
- Parties

Internal teams

- Integration points
- Autonomy

External partners

- Guidelines
- Standards



Partners sell ticket blocks

Corporate buyers purchase in bulk

Offer discounts, box seats, other perks

Service should support both direct and partner sales



Temporal Coupling

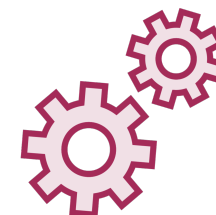
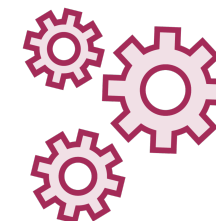
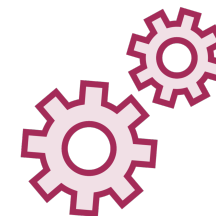


Client is blocked waiting on server

Can allocate threads

Can use promises or tasks

Client Context



Client

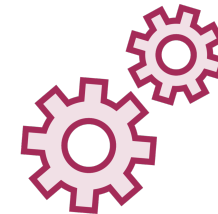
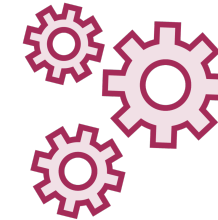
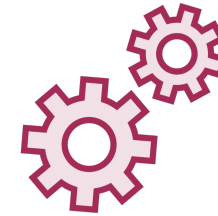
Server



Client Context



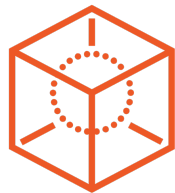
Client



Server



Client Context



Client



Server



Delivery

Short and predictable

Hold client context

Network partition

Timeout

Loss of availability

Retry

Duration

Failures

Mitigations

Processing

Long and variable

Validation

Invariant

Downstream coupling

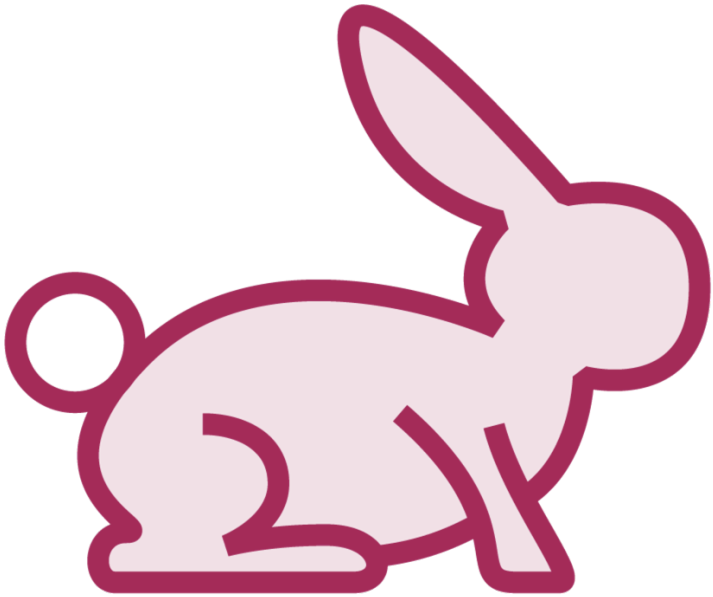
Dead letter queue

Fix upstream

Retry



Asynchronous Protocols



Not related to async/await

Determines the relationship between client and server



Protocols

Synchronous

Response occurs after request is processed

Response conveys information about processing

Report on delivery and processing failures

Asynchronous

Response occurs after delivery

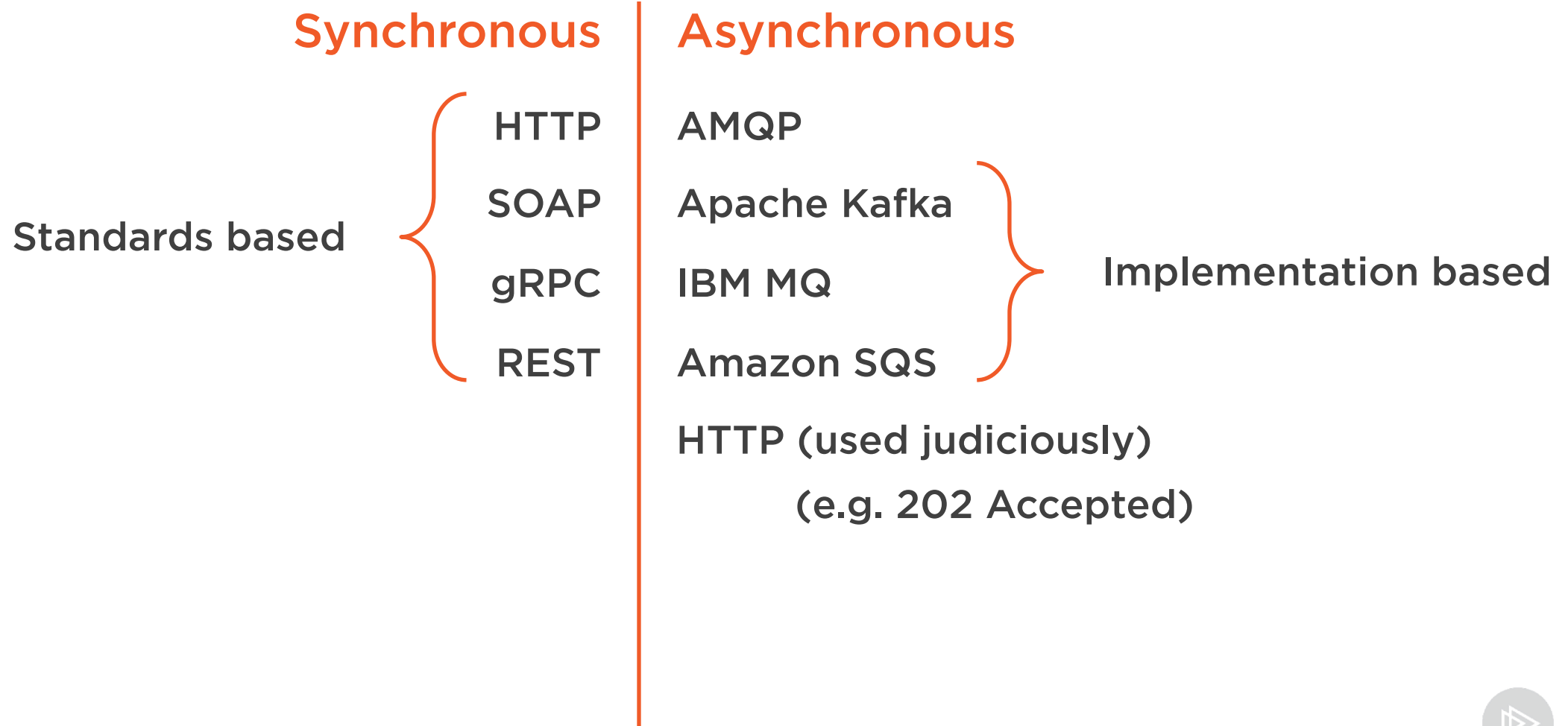
Response cannot convey information about processing

Response cannot report success

Report only on delivery failures



Common Protocols

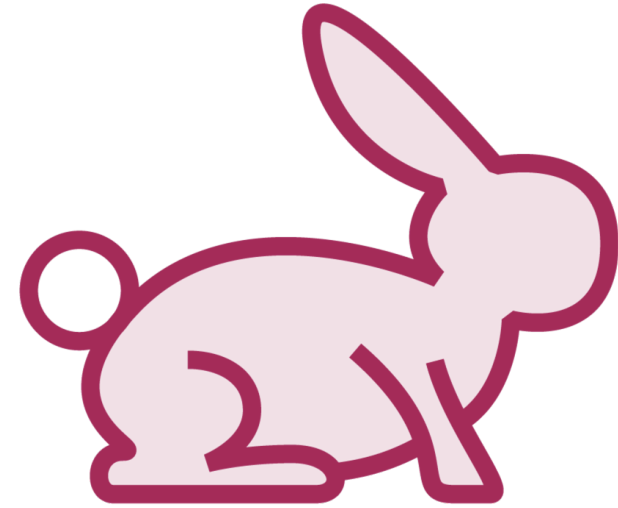


Choosing a Protocol



Synchronous

Provides information
about processing



Asynchronous

Reduces temporal
coupling

Message Categories



Commands

Side-effect
Return no data



Queries

No side-effect
Return data



Events

After side-effect
Expose data



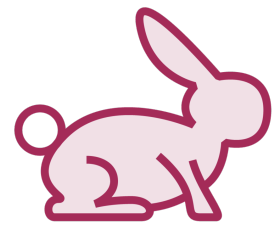


Queries

Processing means
getting state



Synchronous



Asynchronous



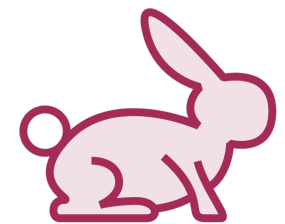
Drop results off
instead of returning



Queries



Synchronous



Asynchronous





Queries

Decoupling is desired

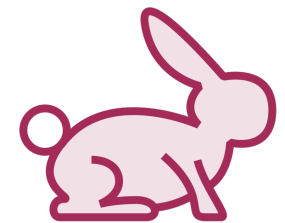
Processing knowledge
is not needed



Events



Synchronous



Asynchronous





Queries



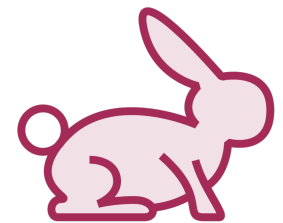
Commands



Events



Synchronous



Asynchronous





Queries

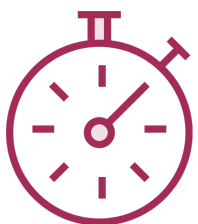
Client needs no
processing information



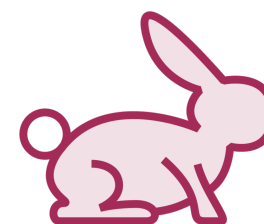
Commands



Events



Synchronous



Asynchronous





Queries

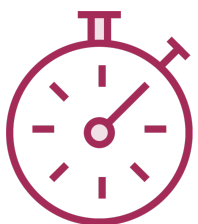


Commands

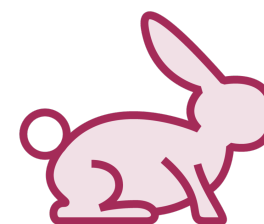
Client needs
processing information



Events



Synchronous



Asynchronous



Purchase Ticket Command



Third-party dependency

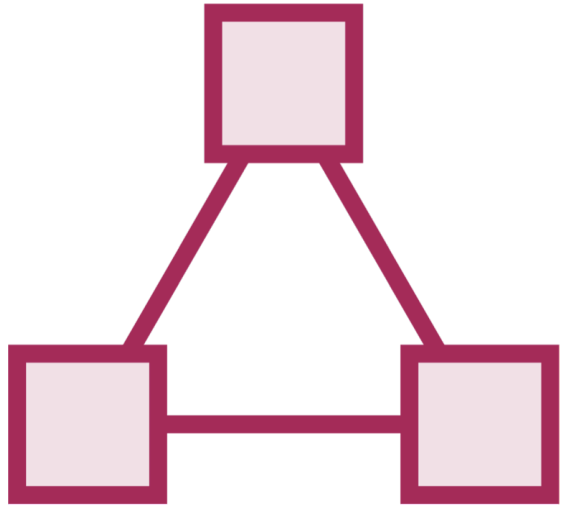
Long time

Unpredictable duration

Many failure conditions

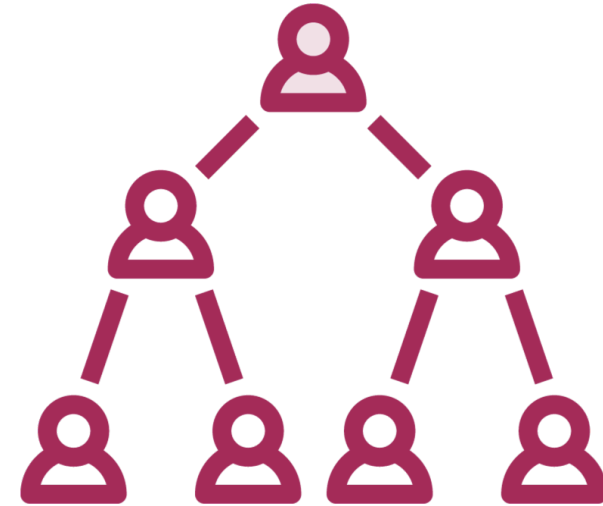
Asynchronous protocol

Aligned Architecture with Organization



Technical structure

Separate services



Organizational structure

Separate teams

Coupling



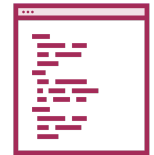
Temporal

Asynchronous protocols



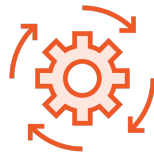
Data

Consistency



Source code

Separate solutions and repositories



CI/CD

Separate pipelines



Versioned Contracts



Projects containing data structures

Additive structure

Published as NuGet packages

Owned by the system of record

Internal NuGet repository

Update dependencies as needed



Semantic Versioning

2.1.0.2106

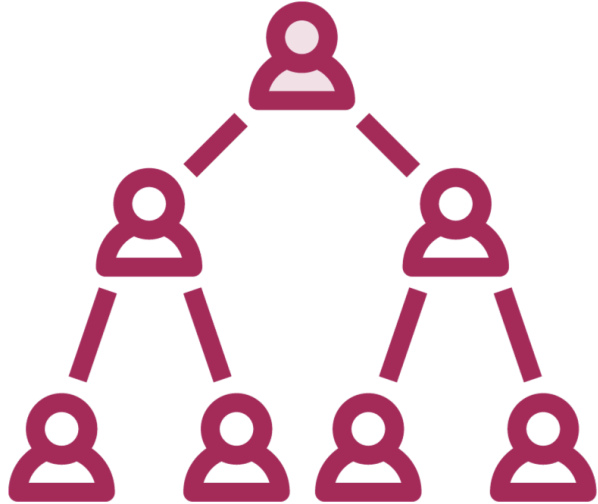
Good for encapsulated libraries

Not good for integration contracts

Favor new services and contracts
to introduce breaking changes



Invoking Business Processes



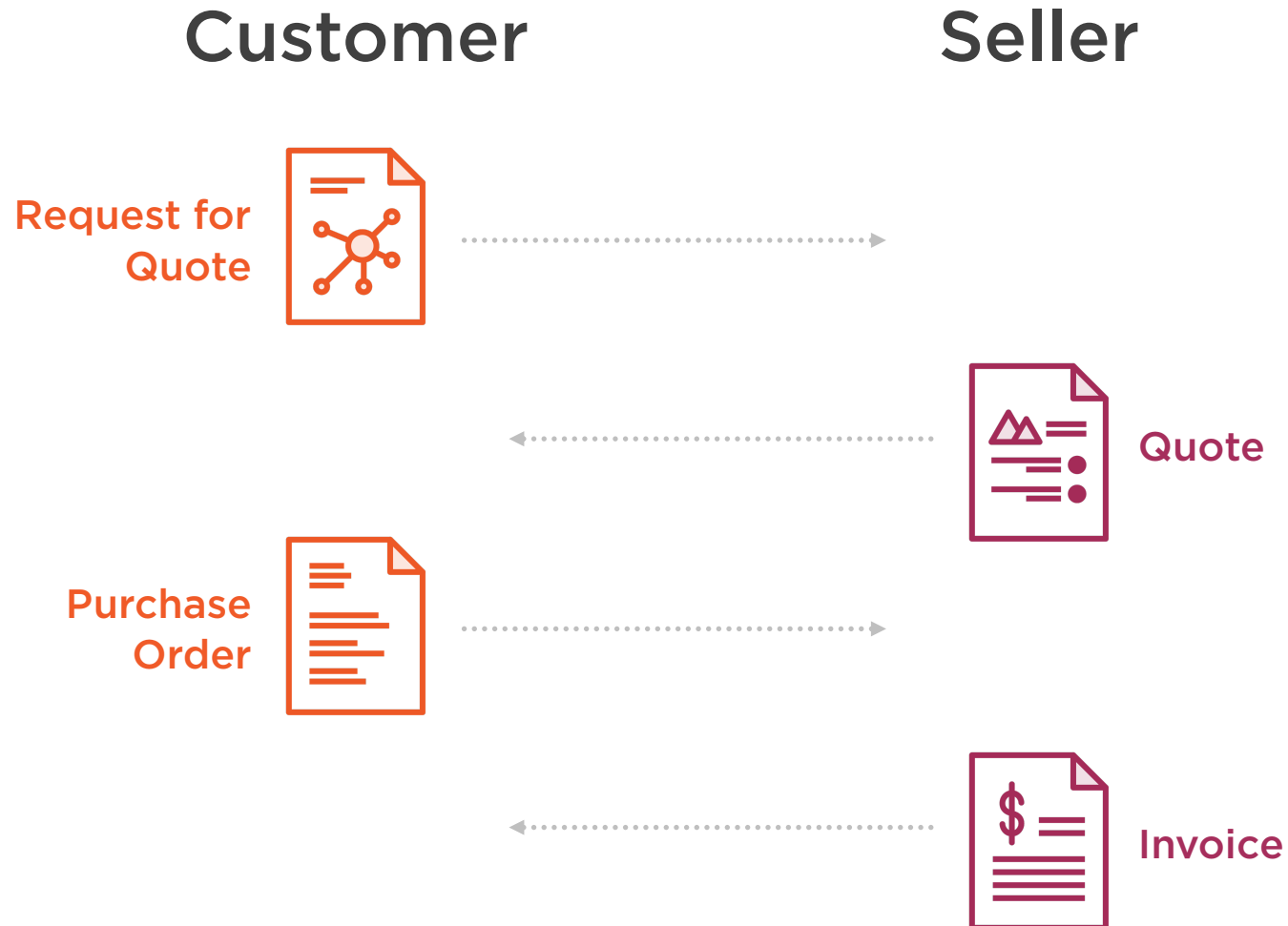
Within an organization



Across organizational boundaries



Transacting Business via Document Exchange





Use document exchange as a template

Start with promotion system

Model sales to external partners

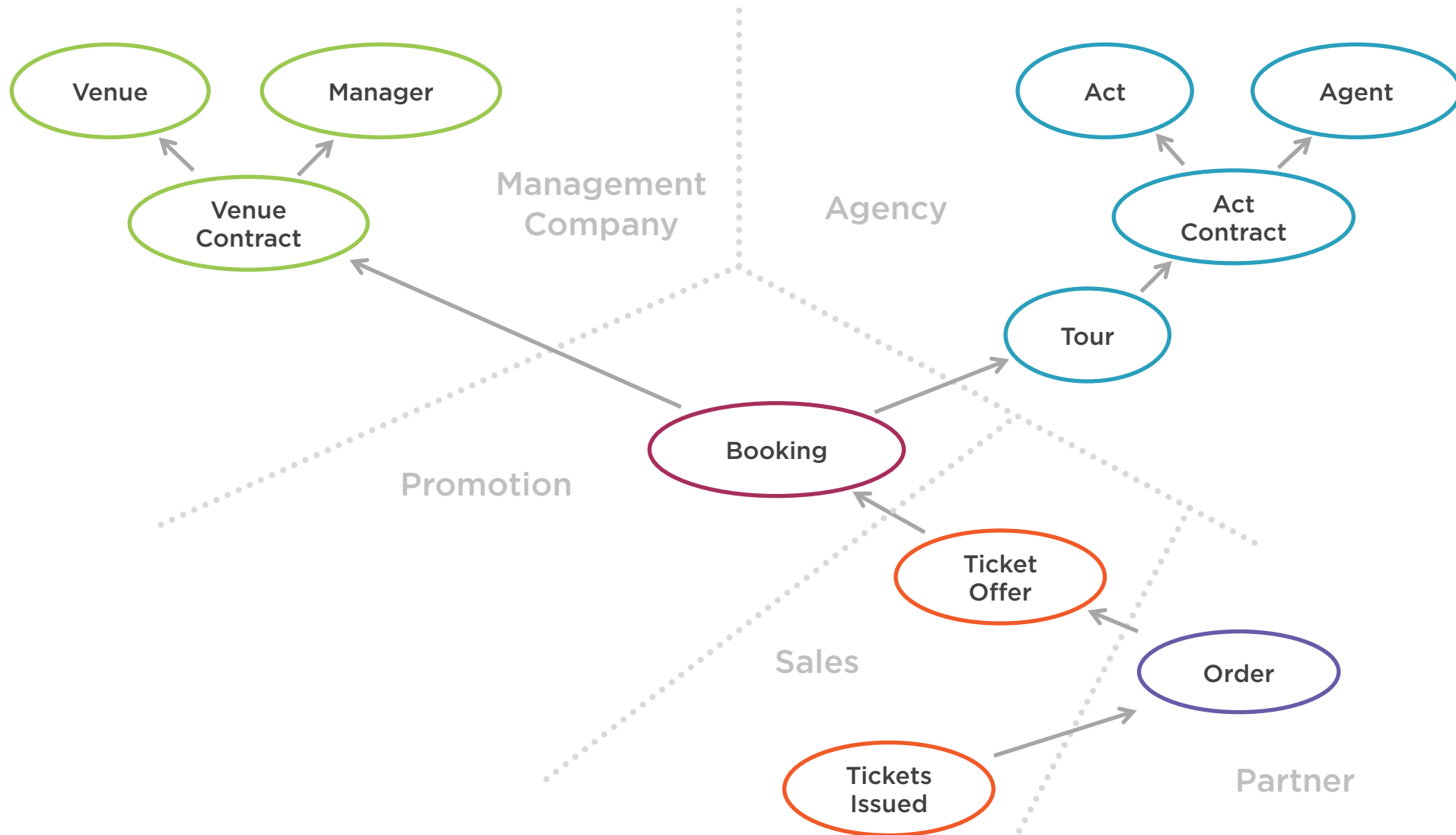
Special pricing

Extra perks

API, not customer website



Historical Model



API Styles

Remote Procedure Call (RPC)

Temporal coupling

Messaging

Implementation-based

Lacks authorization and
validation rules

Asynchronous HTTP

Similar to REST



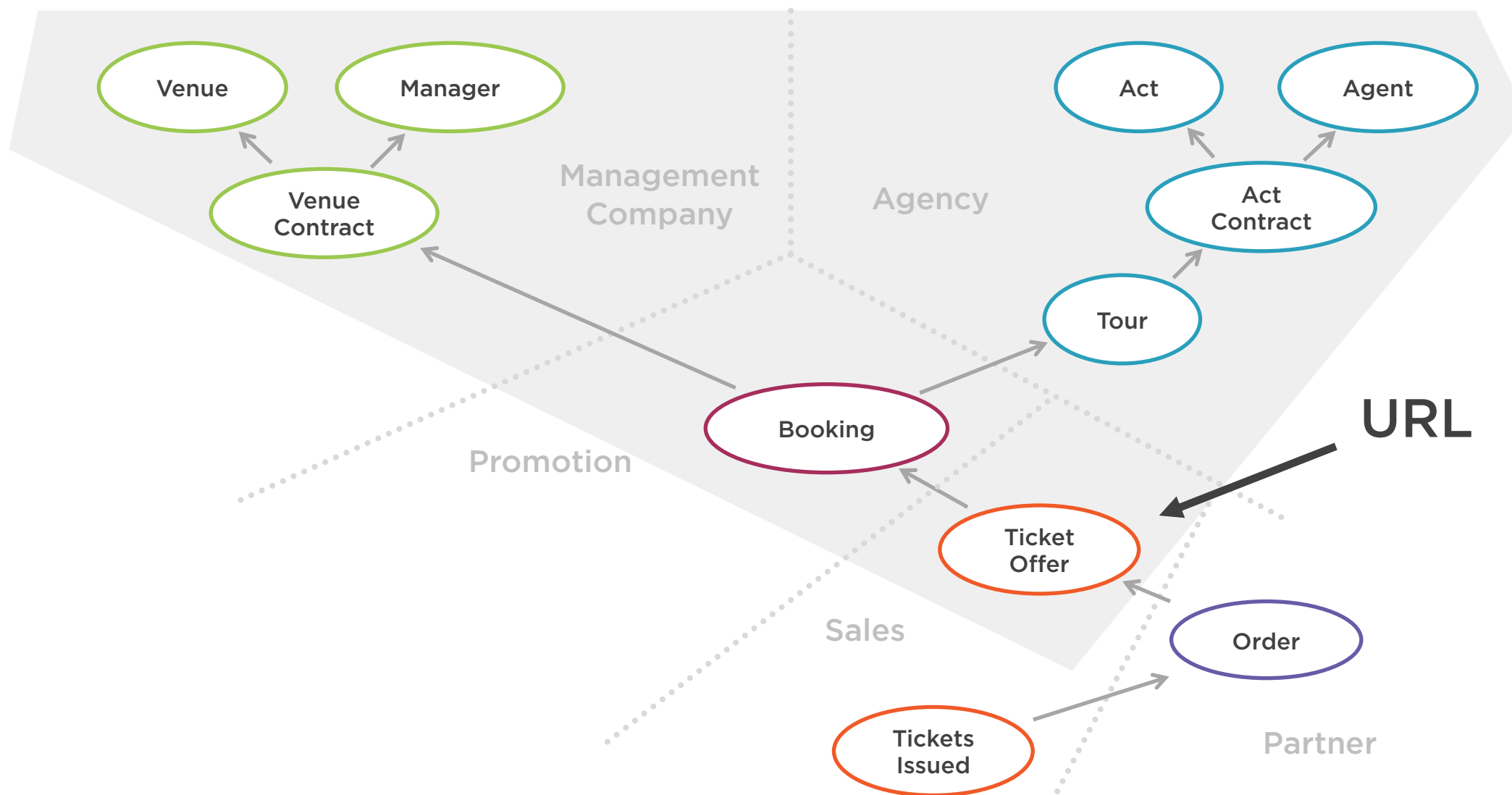
REST



Representational State Transfer
Resources – mutable documents
Uniform Resource Locator (URL)



Historical Model via URL



GET /shows ? **actName**=xxx
& **afterDate**=2021-02-03
& **beforeDate**=2021-05-10

```
[  
  {  
    href: "/shows/{act}/  
           {venue}/{start}",  
    ...  
  },  
  { ... }  
]
```

- ◀ Cannot contain body
- ◀ Cannot change state

◀ Transitive closure of show



Query



Synchronous



POST /shows/{act}/{venue}/{start}/
offers

```
{  
  quantity: 10,  
  section: "Orchestra"  
}
```

```
{  
  href:      "/shows/{act}/  
             {venue}/{start}/  
             offers/{offer}",  
  unitPrice: 289.50,  
  creationDate: "2021-01-05T16:19",  
  expiryDate:  "2021-01-06T16:19"  
}
```

◀ Uniquely identifies offer



Command



Synchronous



POST /shows/{act}/{venue}/{start}/
offers/{offer}/orders

```
{  
  orderGuid: "xxxxx",  
  creditCardPayment: { ... },  
  invoiceRequest: { ... }  
}
```

202 Accepted

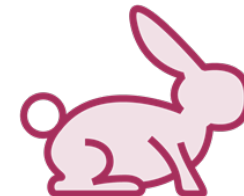
```
{  
  href: "/shows/{act}/{venue}/{start}/  
    offers/{offer}/  
    orders/{order}"  
}
```

- ◀ Client-generated identifier
- ◀ Variable structure

- ◀ Avoid temporal coupling



Command



Asynchronous

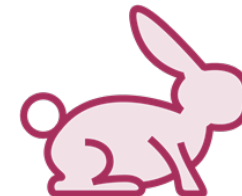


```
GET /shows/{act}/{venue}/{start}/  
offers/{offer}/  
orders/{order}
```

```
{  
  orderStatus: "completed",  
  ticketsIssued: [  
    { ... },  
    { ... },  
    { ... }  
  ]  
}
```



Event



Asynchronous



```
POST /shows/{act}/{venue}/{start}/  
offers/{offer}/orders
```

```
{  
  orderGuid: "xxxxx",  
  creditCardPayment: { ... },  
  invoiceRequest: { ... },  
  webhook: "https://partner/callback"  
}
```

202 Accepted

```
{  
  href: "/shows/{act}/{venue}/{start}/  
    offers/{offer}/  
    orders/{order}"  
}
```

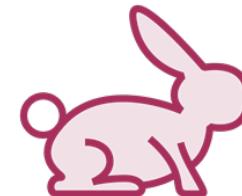
◀ Request push notification



Event



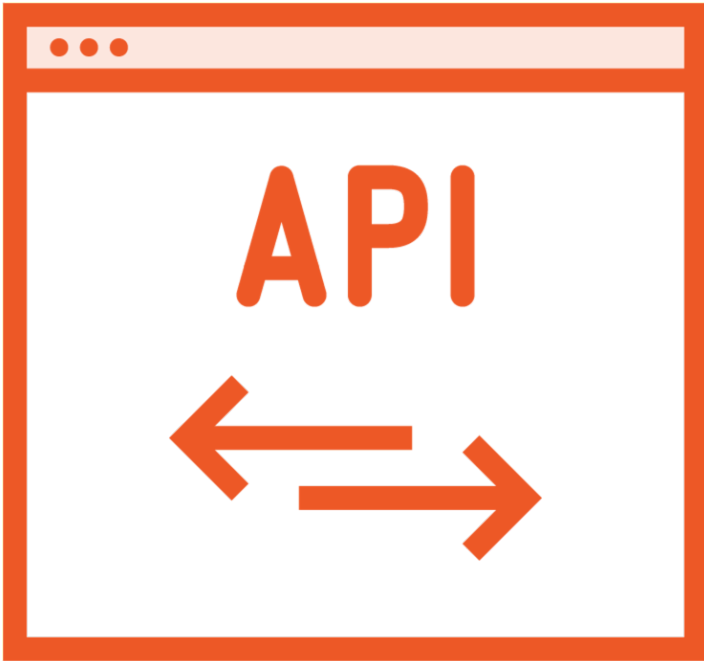
Command



Asynchronous



Document the API



OpenAPI

Postman collections

Reference implementations

API Management

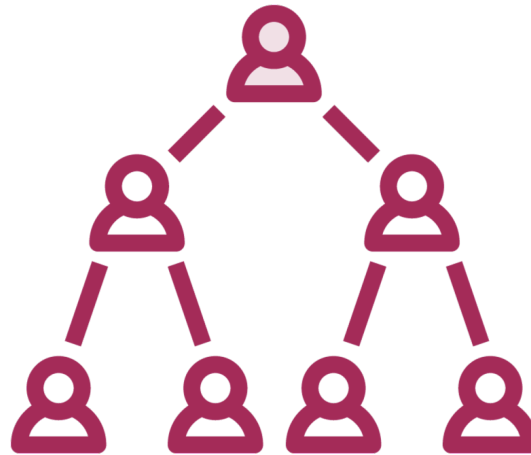


Distributing a Business Process



Temporal coupling

Asynchronous communications
Delivery status,
not processing status



Within the organization

Messaging protocols
Publish internal packages



Across organizational boundaries

Asynchronous HTTP
Document API



Up Next:
Keeping Things Running

